Système d'exploitation : TD4, sémaphores

février 2026 — Pierre Rousselin

Remarque : par soucis de concision, les inclusions des fichiers d'en-tête et certains tests d'appels système sont omis dans les programmes présentés dans ce sujet.

Exercice 1 : Exemples de sémaphores

Dans toutes les questions qui suivent, on suppose donnée la fonction `main`, les globales suivantes, et les déclarations suivantes :

```
void *hop(void *);
void *plop(void *);
void travailler(void);
sem_t sem1;
sem_t sem2;
int res;
int main(void)
{
        pthread_t th_hop, th_plop;
        sem_init(&sem1, 0, X);
        sem_init(&sem2, 0, Y);
        pthread_create(&th_hop, NULL, hop, NULL);
        pthread_create(&th_plop, NULL, plop, NULL);
        pthread_join(th_hop, NULL);
        pthread_join(th_plop, NULL);
        sem_destroy(&sem1);
        sem_destroy(&sem2);
        return 0;
}
```

Les valeurs initiales `X` et `Y` des sémaphores seront données pour chaque question. La fonction `travailler` représente un travail, arbitraire, qui ne modifie ni les sémaphores, ni la variable `res`. Donner le comportement du programme ou, si une situation de compétition ou une étreinte mortelle (*deadlock*, le programme est bloqué) sont possibles.

1. `X = 0`, `Y = 1` et les fonctions `hop` et `plop` sont les suivantes :

```
void *hop(void *arg) {
        travailler();
        sem_wait(&sem2);
        res = 42;
        sem_post(&sem1);
        travailler();
        sem_wait(&sem2);
        res = 93;
        sem_post(&sem1);
        return NULL;
}
void *plop(void *arg) {
        travailler();
        sem_wait(&sem1);
        printf("%d\n", res);
        sem_post(&sem2);
        travailler();
        sem_wait(&sem1);
        printf("%d\n", res);
        sem_post(&sem2);
        return NULL;
}
```

2. Mêmes fonctions avec `X = 1` et `Y = 1`.

3. Mêmes fonctions avec `X = 0` et `Y = 0`.

4. Mêmes fonctions avec `X = 0` et `Y = 2`.

5. Les fonctions `hop` et `plop` sont les suivantes, `X = 1` et `sem2` n'est pas utilisé. La fonction `main` affiche le contenu de `res` après avoir `join` les deux *threads*. Attention, l'incrémentation n'est pas supposée atomique.

```
void *hop(void *arg) {
        for (int i = 0; i < 3; i++) {
                sem_wait(&sem1);
                res++;
                sem_post(&sem1);
        }
        return NULL;
}
void *plop(void *arg) {
        return hop(NULL);
}
```

6. Même chose pour `X = 2`.

7. Même chose pour `X = 0`.

--- * ---

Correction de l'exercice 1 :

1. Après avoir travaillé, `plop` est bloqué sur l'attente de `sem1`, tandis que `hop` n'attend pas sur `sem2` (car il est initialisé à 1), qui passe à `0`, puis met `42` dans `res`, incrémente `sem1`, ce qui réveille `plop`.

Ensuite `plop` affiche le résultat (42) et incrémente `sem2`, ce qui permet à `hop` de mettre `93` dans `res` puis d'incrémenter `sem1`. `plop` peut maintenant décrémenter `sem1` (qui repasse à 0) afficher le résultat (`93`) et incrémenter `sem2` qui repasse à 1.

2. Avec `X = 1` et `Y = 1`, il y a compétition car `plop` n'attend pas que `hop` produise, il est donc possible qu'il affiche `0` dans certains cas, ou bien `42` si `hop` a fini de produire avant la fin du travail de `plop`.

3. Situation de *deadlock* (étreinte mortelle) dès le départ : chaque *thread* est en attente sur son sémaphore, et `main` attend avec `join` la fin des *thread*, qui n'arrive jamais.

4. Encore compétition, mais plus subtile. Comme `Y` vaut `2` initialement, il est possible que `hop` enchaîne les deux productions (`42` puis `93`) avant que `plop` n'affiche le résultat, et donc que `plop` affiche deux fois `93`. Mais il est aussi possible que le programme se déroule « normalement » (`42` puis `93`).

5. Les deux *threads* font la même chose, l'un des deux décrémente `sem1` puis incrémente `res`, et incrémente `sem1`, ce qui réveille l'autre, etc. Cela permet de s'assurer qu'aucun des deux *thread* n'interrompt l'autre pendant l'incrémentation, donc le résultat est bien `6`.

6. Pour `X = 2`, les deux *threads* peuvent incrémenter « en même temps » la variable `res`. Comme l'incrémentation n'est pas atomique, il peut se produire la chose suivante (entre autres possibilités) : `hop` charge dans un registre la valeur de `res` (`0`) pendant que `plop` charge la même valeur (`0`), les deux *threads* écrivent `1` dans `res` qui n'est incrémentée qu'une fois. En lançant `10000` fois le programme, c'est arrivé quelques fois.

7. *Deadlock* (étreinte mortelle), comme dans la question **3.**.

--- * ---

Exercice 2 : A, puis B

On considère le programme suivant :

```c
void boulot(int n);
int main(void) {
        boulot(100000);
        printf("A\n");
        boulot(100000);
        printf("B\n");
}

void boulot(int n) {
        for (int i = 0; i < n; i++)
                ;
}
```

1. Faire faire le `boulot` à des *threads* :

— un nouveau *thread* fait le boulot puis quand il a fini écrit `A` sur le terminal ;

— un autre nouveau *thread* fait le boulot puis quand il a fini écrit `B` sur le terminal.

2. Quel est le risque si le *thread* initial n'attend pas les autres ?

3. L'ordre des affichages est-il fixé à l'avance ?

4. À l'aide d'un sémaphore, faire en sorte que `A` soit toujours affiché avant `B`. Attention, ces *threads* doivent pouvoir travailler en même temps !

5. Reprendre l'exercice avec trois `boulots` et l'écriture de `A`, puis `B`, puis `C`, dans cet ordre.

--- * ---

Correction de l'exercice 2 :

1. Dans ce cas, il risque fort d'atteindre `return 0;` qui met fin au processus entier et le boulot ne sera pas fait !

2. Non, et en pratique c'est vraiment alterné. L'ordre est indéterminé, il y a une situation de compétition (*race condition*) sur l'écriture dans le terminal (en fait dans le tampon mais bon...).

3.
```c
void boulot(int n);
void *thread1(void *arg);
void *thread2(void *arg);

int main(void)
{
        pthread_t th1, th2;
        sem_t A_fini;

        sem_init(&A_fini, 0, 0);
        if (pthread_create(&th1, NULL, thread1, &A_fini) != 0) {
                perror("pthread");
                return 1;
        }
        if (pthread_create(&th2, NULL, thread2, &A_fini) != 0) {
                perror("pthread");
                return 1;
        }
        pthread_join(th1, NULL);
        pthread_join(th2, NULL);
        sem_destroy(&A_fini);

        return 0;
}
void boulot(int n)
{
        int i;
        for (i = 0; i < n; ++i)
                ;
}
void *thread1(void *arg)
{
        sem_t *A_fini = arg;
        boulot(100000);
        printf("A\n");
        sem_post(A_fini);
        return NULL;
}
void *thread2(void *arg)
{
        sem_t *A_fini = arg;
        boulot(100000);
        sem_wait(A_fini);
        printf("B\n");
        return NULL;
}
```

4.
```c
void boulot(int n);
void *thread1(void *arg);
void *thread2(void *arg);
void *thread3(void *arg);

int main(void)
{
        pthread_t th1, th2, th3;
        sem_t AB_finis[2];

        sem_init(AB_finis, 0, 0);
```

```
        sem_init(AB_finis + 1 , 0, 0);

        if (pthread_create(&th1, NULL, thread1, AB_finis) != 0) {
                perror("pthread");
                return 1;
        }
        if (pthread_create(&th2, NULL, thread2, AB_finis) != 0) {
                perror("pthread");
                return 1;
        }
        if (pthread_create(&th3, NULL, thread3, AB_finis) != 0) {
                perror("pthread");
                return 1;
        }
        pthread_join(th1, NULL);
        pthread_join(th2, NULL);
        pthread_join(th3, NULL);
        sem_destroy(AB_finis);
        sem_destroy(AB_finis + 1);

        return 0;
}
void boulot(int n)
{
        int i;
        for (i = 0; i < n; ++i)
                ;
}
void *thread1(void *arg)
{
        sem_t *AB_finis = arg;
        boulot(100000);
        printf("A\n");
        sem_post(AB_finis);
        return NULL;
}
void *thread2(void *arg)
{
        sem_t *AB_finis = arg;
        boulot(100000);
        sem_wait(AB_finis);
        printf("B\n");
        sem_post(AB_finis + 1);
        return NULL;
}
void *thread3(void *arg)
{
        sem_t *AB_finis = arg;
        boulot(100000);
        sem_wait(AB_finis + 1);
        printf("C\n");
        return NULL;
}
```

--- * ---

Exercice 3 : interblocage (*deadlock*)

On considère le programme (idiot) suivant :

```c
void *f1(void *arg);
void *f2(void *arg);
int main(void)
{
        pthread_t th1, th2;
        sem_t sem[2];
        sem_init(sem, 0, 1);
        sem_init(sem + 1, 0, 1);
        pthread_create(&th1, NULL, f1, sem);
        pthread_create(&th2, NULL, f2, sem);

        pthread_join(th1, NULL);
        pthread_join(th2, NULL);
        sem_destroy(sem);
        sem_destroy(sem + 1);

        return 0;
}
void *f1(void *arg)
{
        sem_t *sem = arg;
        sem_wait(sem);
        printf("f1: sem[0] mis à 0\n");
        sem_wait(sem + 1);
        printf("f1: sem[1] mis à 0.\n");
        sem_post(sem + 1);
        sem_post(sem);
        return NULL;
}
void *f2(void *arg)
{
        sem_t *sem = arg;
        sem_wait(sem + 1);
        printf("f2: sem[1] mis à 0\n");
        sem_wait(sem);
        printf("f2: sem[0] mis à 0\n");
        sem_post(sem);
        sem_post(sem + 1);
        return NULL;
}
```

Donner les différents scénarios possibles dans l'exécution de ce programme (on suppose que les affichages sont atomiques, ce qui est faux mais simplifie l'exercice).

--- * ---

Correction de l'exercice 3 :

1. 1er cas : `th1` consomme les ressources des deux sémaphores avant que `th2` ait l'occasion de le faire. Alors est affiché
   ```
   f1: sem[0] mis à 0
   f1: sem[1] mis à 0.
   ```
   puis il ajoute une ressource à chaque sémaphore, réveillant `th2` qui peut verrouiller, puis déverrouiller.
   ```
   f2: sem[1] mis à 0
   f2: sem[0] mis à 0
   ```
   Finalement, les deux *threads* se terminent, réveillant le *thread* initial, qui peut mettre fin au processus.
2. 2ème cas : interblocage. Pendant que `th1` verrouille `sem[0]`, `th2` verrouille `sem[1]`. Chacun est bloqué. Le *thread* initial est lui bloqué dans l'attente de `th1`.
3. 3ème cas : `th2` consomme les ressources des deux sémaphores avant que `th1` ait l'occasion de le faire. Alors est affiché
   ```
   f2: sem[1] mis à 0
   f2: sem[0] mis à 0
   ```
   puis il ajoute une ressource à chaque sémaphore, réveillant `th1` qui peut verrouiller, puis déverrouiller.
   ```
   f1: sem[0] mis à 0
   f1: sem[1] mis à 0.
   ```
   Finalement, les deux *threads* se terminent, réveillant le *thread* initial, qui peut mettre fin au processus.

--- * ---